

Appendix: Artifact Description

PDSW'26 — WarpX / ADIOS2 I/O benchmarking study

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

- C_1 Balanced-workload write-path characterization on Frontier and Perlmutter (weak scaling; multi-TB/s, including Blosc2).
- C_2 ADIOS2 aggregation (TwoLevelShm, EveryoneWritesSerial, DataSizeBased) under extreme writer-side skew.
- C_3 FlattenSteps for lab-frame BTd snapshots from non-contiguous boosted-frame data at negligible write-time and reader-side metadata-stitching overhead.
- C_4 SPAN and AsyncWrite for buffering and compute/I/O overlap under BTd imbalance.
- C_5 Practical ADIOS2 configuration guidelines for WarpX-like PIC codes.

B. Computational Artifacts

- A_1 <https://github.com/guj/artifacts26> — Frontier and Perlmutter build scripts, WarpX inputs, PerformanceRun and repeat/ jobs, log timer extractor, and result CSVs. (Optional Zenodo DOI may be added after acceptance.)

Art.	Contrib.	Related paper elements
A_1	C_1 – C_5	Table I; Table II; Fig. 2 (Perlmutter 3D); Figs. 3–5 (BTd); small- N mean $\pm$ stdev (§II-B, §III-C, §IV-B); §VI-B recommendations

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

A_1 packages inputs, ADIOS2 parameterizations, repeat/ scripts, the log timer extractor, CSVs for Table I and Figs. 2–5, and bpls -l timing scripts and output for Table II. It supports replotting from CSVs and, where allocations permit, re-execution of the WarpX $\rightarrow$ openPMD-api $\rightarrow$ ADIOS2 write path.

Expected Results

- Table I (Frontier 3D; EWS/TLS $\pm$ Blosc2): relative ordering is the primary claim; absolute TB/s varies with Lustre contention.
- Fig. 2 (Perlmutter 3D): TLS above EWS; aggregate >1 TB/s at 512 nodes.
- BTd: FlattenSteps write-time and reader-side metadata-stitching overhead negligible (Table II: bpls -l ≈ 0.12 s for both configs at all scales); SPAN+AsyncWrite highest among tested configs; DataSizeBased preferred when subfile count is constrained under skew.

Expected Reproduction Time (in Minutes)

- **Setup:** 30–120 min (build stack or load site modules).
- **Execution:** ≈ 1 –2 min compute at 8 nodes (Slurm 5–10 min); queue wait is site-dependent. Full scale (2048 Frontier / 512 Perlmutter nodes) needs a leadership allocation and terabytes of scratch (Fig. 2: a 128-node Perlmutter 3D step is already ≈ 10 TB).
- **Analysis:** <15 min to import CSVs and replot in Google Sheets.

Artifact Setup (incl. Inputs)

Hardware

- Frontier (ORNL; primary documented platform): HPE Cray EX; AMD MI250X; Lustre/ClusterStor ORION (≈ 5 TB/s peak write); weak scaling to 2048 nodes; 8 CPU cores/node for I/O. Setup and job scripts in the repo target Frontier.
- Perlmutter (NERSC): GPU nodes; all-flash Lustre (>5 TB/s); weak scaling to 512 nodes (scratch-quota limited); 4 CPU cores/node for I/O. Same stack via scripts/perlmutter/build.setup (CUDA). Example job: warpx_tests/PerformanceRun/BTD/N8/perlmutter.sh.

Node-local NVMe burst buffers were explored for BTd staging but are not required to regenerate the primary CSV/plot artifacts.

Software

- WarpX 25.12 (<https://github.com/BLAST-WarpX/warpx>)
- openPMD-api bundled with that WarpX build (<https://github.com/openPMD/openPMD-api>)
- ADIOS2 2.11, BPFile / BP5 (<https://github.com/ornladios/ADIOS2>)
- Frontier modules (scripts/frontier/bash.setup.env): cmake/3.30.5, craype-accel-amd-gfx90a, rocm, cray-mpich/8.1.31, cce/18.0.1, hdf5/1.14.5-mpi (optional ccache, ninja)
- Perlmutter modules (scripts/perlmutter/build.setup): PrgEnv-gnu, craype-accel-nvidia80, cudatoolkit, gcc-native/13.2, cmake/3.30.2
- Lossless Blosc2 via ADIOS2 (zstd level 1, bit-shuffle, 2048-byte threshold, 6 threads/rank)
- Python 3.6+ for scripts/frontier/extract_time_3.6.py; Google Sheets to import results/*.csv (no matplotlib drivers)

Datasets / Inputs

No external observational datasets. Decks and Slurm scripts live in warpx_tests/; three-trial scripts in repeat/. Per-scale decks: input.n\${NNODES}. Scripts under repeat/ resolve EXE, bpls, and inputs_3d relative to warpx_tests/ (set #SBATCH -A YOUR_PROJECT).

- `inputs_3d/opmd/regular/`, `regular_blosc/`, `regular_h5/`: 3D Cartesian LWFA ($\approx 33\text{M}$ cells/node; ≈ 2.5 GB/node/step).
- `inputs_3d/opmd/btd_*`, `btd_flatten*`: BTD (skew + out-of-order; FlattenSteps variants).
- `PerformanceRun/3D/`, `PerformanceRun/BTD/`: Frontier jobs (plus BTD/.../perlmutter.sh).
- `repeat/`: same pattern, by config (`bp/`, `bp_async/`, `bp_flatten/`, `h5/`); submit each script three times for $N=8-32$ (`repeat/README.md`).
- `read_metadata/`: `browse_flatten.sh` and `browse_default.sh` run `bpls -l 3` $\times$ per snapshot on a login node; `out_flatten` and `out_default` contain raw timing output (36 trials each); `analyze.py` reproduces Table II.

Paper numbers are in `results/`:

File	Paper
<code>frontier_3d_throughput.csv</code>	Table I
<code>3D-perlmutter.csv</code>	Fig. 2
<code>fig_btd_b_span_async_frontier.csv</code>	Fig. 3
<code>fig_btd_c_agg_perlmutter.csv</code>	Fig. 4
<code>fig_btd_d_subfiles_perlmutter.csv</code>	Fig. 5
<code>repeat_runs_raw.csv</code> , <code>repeat_summary.csv</code>	$\text{mean} \pm \text{stdev}$
<code>read_metadata/</code>	Table II

- 4) **Aggregate.** Write times and sizes into `results/*.csv`; replot in Google Sheets.

Artifact Execution

T_1 build/load $\rightarrow T_2$ run WarpX $\rightarrow T_3$ `extract_time_3.6.py` plus file sizes into CSV $\rightarrow T_4$ Google Sheets. Parameters: node count, aggregation, NumSubFiles, FlattenSteps, SPAN, AsyncWrite, Blosc2. Frontier $N=8-32$ configs were repeated

See `warpx_tests_INDEX.md`, `repeat/README.md`, `results/README.md`.

Installation and Deployment

Same tags on both sites (ADIOS2 2.11, WarpX 25.12, bundled openPMD-api, WarpX_openpmd_internal=ON; override with ADIOS2_TAG, WARPX_TAG). Choose scratch WORKDIR (no hard-coded user path).

- 1) **Build.** Frontier (HIP): `export WORKDIR=/lustre/orion/<proj>/scratch/<u>/<exp>; source scripts/frontier/bash.setup.env; ./scripts/frontier/build.setup.` Perlmutter (CUDA): `export WORKDIR=/pscratch/sd/<u>/<exp>; ./scripts/perlmutter/build.setup` (loads NERSC modules).

- 2) **Point and submit.** In `warpx_tests/`, symlink `EXE` $\rightarrow$ WarpX and `bpls` $\rightarrow$ ADIOS2 `bpls`; set `#SBATCH -A`. Submit job scripts under `PerformanceRun/` (scale-out) or `repeat/` ($3 \times$ small- N repeats).

- 3) **Extract.** From the parent of each `job_id/`: `python3scripts/frontier/extract_time_3.6.pyjob_id` $\rightarrow$ TotalTime, IOTime. Option 1 I/O time: $\text{TotalTime}_{\text{I/O}} - \text{TotalTime}_{\text{nullcore}}$ when nullcore runs exist; **DataSize** is the on-disk output size.

three times (`repeat/`); larger scales and Blosc are single-trial, with reliability via cross-scale consistency (§III-C).

Artifact Analysis (incl. Outputs)

CSV throughput vs. nodes plus small- N $\text{mean} \pm \text{stdev}$; plots of Figs. 2–5 from those CSVs; `bpls -l` timings for Table II. Compare to Table I, Table II, and Figs. 2–5 on relative trends; document filesystem contention as in §VI-C.